MAWC (Multi-Agent Warehouse Coordination) — Complete Technical Specification

1. ENVIRONMENT SPECIFICATION

1.1 Arena Dimensions

- Size: 20.0 m (W) × 12.0 m (L) × 3.0 m (H)
- Floor: Single-layer concrete, friction coefficient = 0.8
- Walls: 0.2 m thick, 3.0 m height, placed at boundaries

1.2 Shelf Layout (4 Double-Sided Rows)

Row	Center Y (from S wall)	X Range	Length	Width	Height
1	2.5 m	3.5–7.5 m	4.0 m	0.6 m	1.2 m
2	5.5 m	4.0–8.0 m	4.0 m	0.6 m	1.2 m
3	8.5 m	3.5–7.5 m	4.0 m	0.6 m	1.2 m
4	11.0 m	5.0–9.0 m	4.0 m	0.6 m	1.2 m

- Aisle width between rows: 1.5 m (center-to-center: 3.0 m)
- Shelf material: Solid collision mesh in PyBullet

1.3 Depot Zone

- Location: Southwest corner
- Coordinates: (1.0, 1.0) to (3.0, 3.0) — 2m × 2m square
- Visual: Green floor marker

1.4 Resource Positions (12 items)

All resources are 0.3m × 0.3m × 0.3m red cubes, placed in aisles on floor level.

ID	Coordinates (x, y)	Aisle Location
R1	(4.0, 2.5)	Row 1 aisle
R2	(6.5, 2.5)	Row 1 aisle
R3	(9.0, 2.5)	Row 1 aisle
R4	(5.0, 5.5)	Row 2 aisle
R5	(7.5, 5.5)	Row 2 aisle
R6	(10.0, 5.5)	Row 2 aisle
R7	(4.5, 8.5)	Row 3 aisle
R8	(8.0, 8.5)	Row 3 aisle
R9	(6.0, 11.0)	Row 4 aisle
R10	(9.5, 11.0)	Row 4 aisle
R11	(12.0, 11.0)	Row 4 aisle
R12	(14.5, 11.0)	Row 4 aisle

Note: Coordinates randomized during training for generalization.

1.5 Start Positions

All 4 robots spawn side-by-side ("baju baju") in a 0.8m radius cluster centered at (1.5, 1.5) within the depot zone.

- Random small offsets: ±0.15m position, ±0.2 rad orientation
- Non-overlapping: minimum 0.5m separation enforced

---

2. ROBOT URDF SPECIFICATIONS

2.1 Chassis

- Type: Differential drive mobile base
- Dimensions: 0.4 m (L) × 0.3 m (W) × 0.25 m (H)
- Mass: 15 kg
- Wheel base: 0.26 m (distance between wheel centers)
- Wheel radius: 0.05 m
- Wheel width: 0.04 m
- Max linear velocity: 0.5 m/s
- Max angular velocity: 1.0 rad/s
- Max linear acceleration: 1.0 m/s²
- Max angular acceleration: 2.0 rad/s²

2.2 LiDAR

- Type: 2D planar LiDAR (simulated as ray-cast array)
- Field of View: 270° (front-facing, -135° to +135°)
- Range: 0.1 m to 10.0 m
- Resolution: 720 rays (0.375° per ray)
- Update rate: 10 Hz
- Noise model: Gaussian = 0.01 m + 1% of range

2.3 Manipulator (Optional for Simulation)

- Type: 5-DOF articulated arm
- Reach: 0.8 m
- Base mount: Top-center of chassis
- End effector: Parallel jaw gripper
- Jaw opening: 0.0–0.15 m
- Max payload: 5 kg
- Pick zone: Ground level only (z = 0.15–0.35 m)

2.4 Communication

- Type: Discrete broadcast radio
- Range: Unlimited (global broadcast)
- Bandwidth: 1 token per 0.5 s (2 Hz)
- Vocabulary size: K = 16 tokens
- No packet loss, no latency

2.5 URDF Link/Joint Structure

```
base_link (chassis)
├── left_wheel_joint → left_wheel
├── right_wheel_joint → right_wheel
├── caster_front_joint → caster_wheel
├── caster_rear_joint → caster_wheel
├── lidar_mount_joint → lidar_link
│   └── lidar_optical_frame
└── arm_base_joint → arm_link_1
    ├── arm_joint_1 → arm_link_2
    ├── arm_joint_2 → arm_link_3
    ├── arm_joint_3 → arm_link_4
    ├── arm_joint_4 → arm_link_5
    └── gripper_base_joint → gripper_link
        ├── left_finger_joint → left_finger
        └── right_finger_joint → right_finger
```

---

3. REWARD SPECIFICATION (90/10 Split)

3.1 Shared Rewards (90% weight)

Applied identically to all 4 agents.

Event	Reward	Frequency
All 12 resources delivered	+100.0	Once per episode
Per successful delivery (any bot)	+10.0	Per delivery
Makespan bonus	$+50 \times (T\_max - T\_actual) / T\_max$	Once per episode
Collision (any pair)	−5.0	Per collision event
Time penalty	−0.05 per step	Every step

Makespan Bonus Example:

- $T\_max = 500$ s, $T\_actual = 200$ s
- Bonus $= 50 \times (500-200)/500 = +30.0$

3.2 Individual Rewards (10% weight)

Applied per-agent.

Event	Reward	Frequency
Successful personal pickup	+1.0	Per pickup
Successful personal delivery	+2.0	Per delivery
Idle penalty ($v < 0.1$ m/s, not at depot)	−0.02 per step	Every step

Table 4 – continued

Event	Reward	Frequency
Replanning penalty (A* re-triggered)	−0.1	Per replan
Invalid action penalty	−0.5	Per invalid action

3.3 Total Reward Calculation

```
R_total_i = 0.90 × R_shared + 0.10 × R_individual_i
```

3.4 Reward Shaping (Optional Curriculum)

- Early training (episodes 0–50k): Add +2.0 for "claiming a resource" (encourages commitment)
- Mid training (50k–150k): Remove claim bonus, keep delivery bonuses
- Late training (150k+): Full reward structure with makespan bonus only

---

4. OBSERVATION SPACE (Per Agent, at 2 Hz)

Component	Shape	Description
self_pose	(3,)	(x, y,) from EKF, continuous
self_velocity	(2,)	(v,) current command
self_carrying	(1,)	Binary: 0=empty, 1=carrying
other_poses	(9,)	Flattened (x, y,) of other 3 bots
other_carrying	(3,)	Binary carrying status of others
resource_status	(12,)	0=available, 1=claimed-by-me, 2=claimed-by-other, 3=delivered
depot_vector	(2,)	(dx, dy) relative to self
map_embedding	(64,)	CNN embedding of static warehouse map
comm_input	(32,)	Mean-pooled embedding of received tokens
time_elapsed	(1,)	t / T_max, normalized
planned_path_cost	(1,)	A* path cost to current target

Total observation dimension: 132

---

5. ACTION SPACE (Continuous + Discrete Hybrid)

Policy runs at 2 Hz. All outputs are from a single shared neural network.

Head	Space	Activation	Description
target_x	Continuous, [−1, 1]	Tanh	Scaled to arena x-bounds (0–20m)
target_y	Continuous, [−1, 1]	Tanh	Scaled to arena y-bounds (0–12m)

Table 6 – continued

Head	Space	Activation	Description
target_resource	Discrete(13)	Gumbel-Softmax	0–11=claim resource, 12=wait/depot
communicate	Discrete(16)	Gumbel-Softmax	Broadcast token m {0,⋯,15}
yield_priority	Discrete(2)	Gumbel-Softmax	0=assert, 1=yield

Note: The continuous (target_x, target_y) are used ONLY when target_resource=12 (free navigation). When target_resource is 0–11, the target is automatically set to that resource's coordinates.

---

6. NAVIGATION STACK

6.1 EKF (Extended Kalman Filter)

State: x = [x, y, , v_bias, _bias]

Prediction (every 0.1 s):

```
x_k|k-1 = f(x_k-1, u_k)
P_k|k-1 = F_k · P_k-1 · F_kᵀ + Q_k

where:
  u = [v_cmd, ω_cmd]ᵀ (wheel odometry)
  f(x,u) = [x + (v·cosθ)·dt, y + (v·sinθ)·dt, θ + ω·dt, v_bias, ω_bias]ᵀ
  v = v_cmd − v_bias
  ω = ω_cmd − ω_bias
```

Update (when LiDAR scan available, 10 Hz):

```
Measurement model: z = h(x) = [range_i, bearing_i] for each LiDAR ray
H = Jacobian of h w.r.t. x
K = P·Hᵀ·(H·P·Hᵀ + R)⁻¹
x_k = x_k|k-1 + K·(z_observed − z_predicted)
P_k = (I − K·H)·P_k|k-1
```

Key Parameters:

- Process noise Q: diag(0.001, 0.001, 0.0005, 0.0001, 0.0001)
- Measurement noise R: diag(_r², _ ²) = diag(0.01, 0.001)
- Initial covariance P : diag(0.1, 0.1, 0.05, 0.01, 0.01)

6.2 A* Path Planning

Grid: 0.1 m resolution, 200 × 120 cells **Connectivity:** 8-connected (allows diagonal movement) **Heuristic:** Euclidean distance × 1.001 (admissible) **Cost function:**

```
cost(cell) = 1.0 + 10.0×occupancy + 2.0×proximity_to_other_agent
```

Replanning triggers:

1. Target resource changed by RL policy
2. Dynamic obstacle detected within 1.0 m of current path
3. Every 2.0 s (heartbeat replan)
4. Path cost deviation > 20% from expected

6.3 DWA (Dynamic Window Approach)

Runs at: 10 Hz **Input:** Global path waypoints from A* **Output:** (v,) command for next 0.1 s

Velocity sampling space (dynamic window):

```
v ∈ [v − v̇ · dt, v + v̇ · dt] ∩ [0, v_max]
ω ∈ [ω − ω̇ · dt, ω + ω̇ · dt] ∩ [−ω_max, ω_max]
```

Trajectory simulation (3.0 s horizon, dt=0.1 s):

```
For each (v, ω) candidate:
  Simulate forward: x_t+1 = x_t + v · cos(θ) · dt, etc.
  Score = α · goal_progress + β · obstacle_clearance + γ · velocity + δ · heading_align
```

Scoring weights:

- (goal progress): 0.5
- (clearance): 0.3 — penalizes proximity to obstacles
- (velocity): 0.15 — prefers higher speeds
- (heading): 0.05 — aligns with path direction

Obstacle clearance:

```
clearance = min_distance_to_obstacle(trajectory)
if clearance < 0.3m: reject trajectory
```

6.4 Pure Pursuit Controller

Input: Best trajectory from DWA **Output:** Left/right wheel velocities (v_L, v_R)

```
look_ahead_distance = 0.5 m
target_point = point on path at look_ahead_distance
curvature = 2 × y_error / look_ahead_distance²

v_L = v − ω × (wheel_base / 2)
v_R = v + ω × (wheel_base / 2)
```

---

7. RL ARCHITECTURE

7.1 Algorithm: MAPPO with CTDE

- **Actor:** Decentralized. Input = local obs (132-dim). Output = action distribution.
- **Critic:** Centralized. Input = global state (all poses, all claims, true map). Output = V(s).

- **Parameter Sharing:** All 4 agents share identical actor weights.

7.2 Network Architecture

```
Input: obs (132)
|
├─> [Map Encoder] ──> 64D (CNN on static map patch)
├─> [Comm Encoder] ──> 32D (MLP on received tokens)
├─> [State Encoder] ──> 64D (MLP on pose, velocity, resource status)
|
└─> Concatenate (160D)
    |
    ├─> [Actor Core] ──> 128D ──> [Policy Heads]
    |   ├─> target_x: Tanh → scale
    |   ├─> target_y: Tanh → scale
    |   ├─> target_resource: 13-way softmax (Gumbel-Softmax train)
    |   ├─> communicate: 16-way softmax (Gumbel-Softmax train)
    |   └─> yield_priority: 2-way softmax
    |
    └─> [Critic Core] ──> 128D ──> V(s) scalar
```

7.3 Training Hyperparameters

Parameter	Value
Learning rate (actor)	3×10
Learning rate (critic)	1×10^{3}
Discount	0.99
GAE	0.95
Clip	0.2
Entropy coef	0.01
Value loss coef	0.5
Batch size	4096 steps
Mini-batch size	512
PPO epochs	10
Communication vocab K	16

---

8. COMPLEXITY ANALYSIS

8.1 Computational Complexity (Per Step)

Component	Complexity	Time @ 2.4 GHz
EKF Prediction	O(1)	~0.05 ms
EKF Update (720 rays)	O(n) where n=720	~0.3 ms
A* Planning (200×120 grid)	O(b^d) O(10) worst case	~2–5 ms
DWA (100 candidates × 30 steps)	O(3000)	~1 ms

Table 8 – continued

Component	Complexity	Time @ 2.4 GHz
Neural Network Forward	O(d²) O(10)	~0.5 ms (GPU)
Total per robot		**~5–8 ms**
Total 4 robots		**~20–32 ms**

Conclusion: Easily runs at 2 Hz (500 ms budget) with 15× margin.

8.2 Learning Complexity

Aspect	Complexity	Mitigation
Joint action space	13 × 16 × 2 = 416 per agent; 416 3×10^{1} joint	Parameter sharing + CTDE
Credit assignment	High (shared reward)	90/10 split + GAE
Non-stationarity	High (4 concurrent learners)	Parameter sharing stabilizes
Communication grounding	Open problem	Discrete tokens + curriculum
Sample efficiency	~10 steps needed	Use parallel envs (8–16)

---

9. OPTIMIZATION STRATEGIES

9.1 Movement Optimization

1. **Path caching:** Cache A* results for common origin-destination pairs.
2. **Lazy replanning:** Only replan when obstacle distance < 0.8m, not on every LiDAR update.
3. **Velocity scheduling:** DWA reduces speed to 0.2 m/s near resources/depot, 0.5 m/s in open aisles.
4. **Diagonal preference:** A* 8-connected grid naturally prefers diagonal cuts across open space.
5. **Collision prediction:** Predict other agents' positions 1.0 s ahead; add as dynamic obstacles in DWA.

9.2 Learning Optimization

1. **Curriculum Learning:** Start with 4 resources → 8 → 12.
2. **Reward shaping:** Early claim bonus, later remove.
3. **Communication dropout:** Randomly zero-out received messages during training (10% probability) to force robust protocols.
4. **Population-based training:** Train 4 populations with different entropy coefficients; migrate best weights.
5. **Prioritized experience replay:** Weight experiences where makespan was close to optimal.

9.3 Simulation Optimization

1. **PyBullet:** Use `setRealTimeSimulation(0)` and step manually.
2. **Parallel envs:** Run 16 warehouse instances on 4 CPU cores.
3. **LiDAR batching:** Batch all 4 robots' ray-casts into single PyBullet call.
4. **GPU inference:** Batch all 4 observations into single forward pass.

10. COMPARISON: OLD PLAN vs NEW PLAN

Aspect	Old Plan (HiveMind Foraging)	New Plan (MAWC Warehouse)	Winner
Observability	Partial (LiDAR only)	Full (known map + positions)	New
Task clarity	Exploration + foraging	Coordination + scheduling	New
Scalability	Hard (POSG)	Easier (known state)	New
Real-world gap	Large (emergent roles)	Small (warehouse logistics)	New
Research novelty	High (emergent roles)	Medium (emergent comm only)	Old
Implementation complexity	Very High	Moderate	New
Training stability	Low (non-stationary)	High (full obs)	New
CV requirement	Optional (RGB for scouts)	Not required	New
Makespan optimization	Not addressed	Core objective	New

Recommendation: Proceed with the **New Plan (MAWC)**. It is more tractable, directly applicable to real warehouses, and the emergent communication problem is still scientifically valid (distributed auction without central controller).

---

11. OPEN QUESTIONS & CONFIRMATIONS NEEDED

1. **URDF Files:** You mentioned attaching URDF files, but none were uploaded. Please provide your robot URDF so I can verify joint names, link dimensions, and sensor mounts.
2. **Shelf Geometry:** Are shelves single-sided or double-sided? I assumed double-sided back-to-back. Confirm.
3. **Resource Height:** Are resources on the floor (ground-level pick) or on shelf tiers? I assumed floor-level for single-layer operation.
4. **Dynamic Obstacles:** Should humans/falling boxes be simulated as dynamic obstacles, or is the environment static after spawn?
5. **Communication Range:** I assumed unlimited range. Should there be a cutoff (e.g., 5m)?
6. **Collision Handling:** On collision, do robots bounce, stop, or get penalized and continue? I assumed penalty + continue.
7. **Iteration 1 Assignment:** Should Iteration 1 use greedy nearest-neighbor, round-robin, or random assignment?

8. **Gripper Simulation:** Do you need full PyBullet constraint-based grasping, or simple "within 0.2m + pickup action = attached"?
9. **Evaluation Metric:** Primary metric is makespan. Should we also track total distance traveled, collision count, and communication entropy?
10. **Real Hardware Target:** Is this purely simulation, or do you plan to deploy on TurtleBot3 / Clearpath / custom hardware?